



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

ELECTENG 209

Embedded Software Design

Interrupts

Duleepa J Thrimawithana

Learning Objectives

- Why are interrupts important?
- Understanding the differences between interrupts and polling
- Why do we need interrupts in the project?
- Understanding Interrupt Service Routine (ISR)
- How are interrupts implemented by the microprocessor?
- Learning to configure and use interrupts in the ATmega328P
- Understanding RAM allocation
- Understanding how the stack, heap and data segment spaces are used
- Understanding context switching

Lecture Quiz

- The lecture quiz is now available on Canvas
 - Quiz is available for 3 days and allows 3 attempts
 - Best of the 3 attempts taken as the final score

Input/Output Events

How to Handle I/O Events?

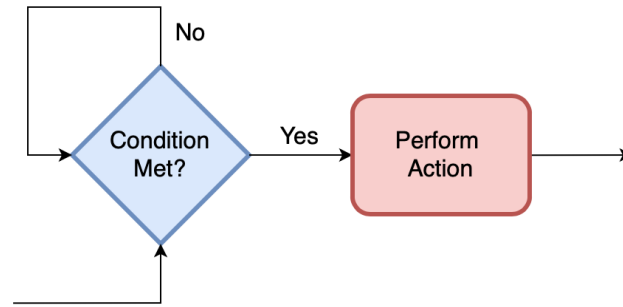
I/O Events

- I/O peripherals enable the microprocessor in an MCU to interface with external electrical signals
 - So far we have looked at GPIO, UART, ADC and Timer peripherals
- The peripherals often take considerable amount of time to complete a given task
 - The time taken can be many system clock cycles and during this time the microprocessor can be executing code
 - For example, a UART transmitting at a low baud rate takes a considerable amount of time to send a byte of data
- The peripherals also have to respond to external signals which are non-deterministic (i.e. we don't know when they will occur)
 - When we are trying to detect such signals, the microprocessor have to communicate regularly with the peripherals checking to see if they have occurred
 - For example, when checking for a push-button press, we have to regularly scan the GPIO pin it is connected at

Responding to I/O Events

- So far we used the polling method to respond to events (e.g. a peripheral completing a task or responding to an external signal)
 - When polling, which is also called “blocking” or “busy waiting”, the microprocessor waits to respond to the event
- Waiting for peripherals to complete tasks and responding to non-deterministic external signals keep the microprocessor unnecessarily busy
- To be efficient, the microprocessor needs to execute code in response to these events at the correct time rather than waiting for them to occur
 - For example, write a byte of data to UART transmit buffer (UDR0) when it is just emptied rather than waiting it to be empty
- The second method to respond to these events, is to use interrupts
 - Interrupts enable the microprocessor to respond to events at the correct time rather than waiting for them to occur

Polling



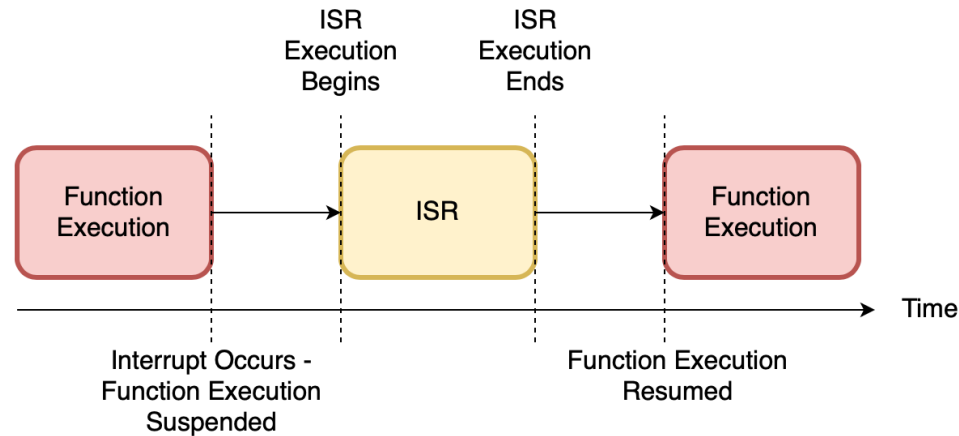
```
void poll(void){  
    while (!condition) {  
        ;  
    }  
    perform_action()  
}
```

- So far we used polling when we interacted with the GPIOs, UART, ADC and timer
- We executed code continuously in the form of a loop to check the state of a given register bit
 - With the UART we checked UDRE0, with ADC we checked ADIF and so on
- The register bits were checked as part of the normal program execution while halting the execution of the rest of the program
 - Once the appropriate state was detected, the action was taken and continued with the rest of the program

Benefits & Drawbacks of Polling

- Benefits of polling include
 - Simple to implement, and does not require any hardware
 - The processor is always in a known state when executing code, so race conditions and variable states do not have to be managed
- Drawbacks of polling include
 - If multiple peripherals require checking, it must do them in a round-robin fashion adding a delay between each check of a given peripheral
 - If two peripherals require servicing, they will be done in the order they are checked, which may not be the order of priority
 - Most of the time, the checking does not result in any work being done, so this processor time has been wasted
 - It can be very difficult to get the processor to run other code in between polling all of its peripherals

Interrupts



- Interrupts were introduced explicitly to handle events
- When an event triggers an interrupt, the normal program execution is halted, and a special function is executed
 - This special function is called the Interrupt Service Routine (ISR)
- Once the ISR has completed, the processor returns to the code it was executing before the interrupt occurred

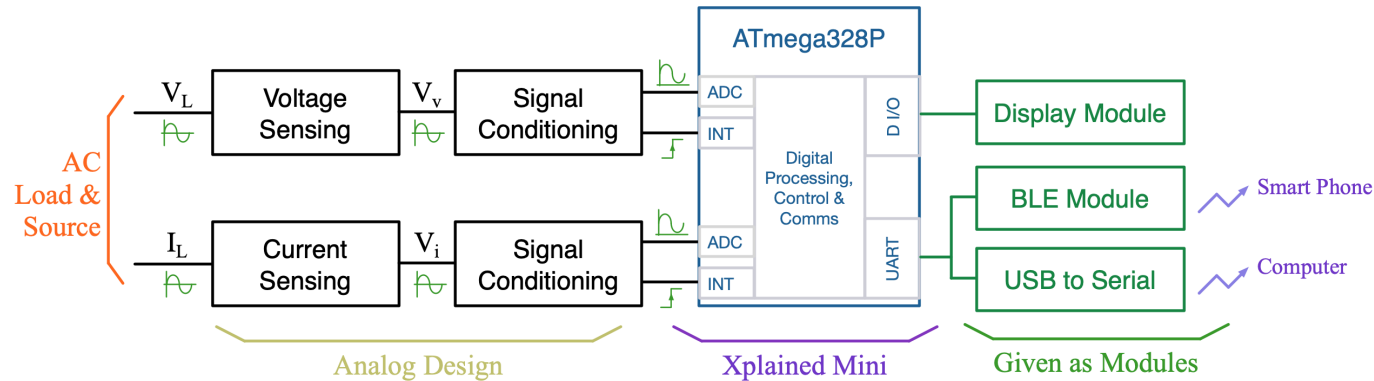
Benefits & Drawbacks of Interrupts

- Benefits of interrupts include
 - Interrupts can respond to an event almost immediately
 - The interrupts can be given priorities to make sure the most important events are always responded to quickly
 - The processor can be performing useful work while waiting for an interrupt to occur
- Drawbacks of interrupts include
 - Extra hardware is required to implement interrupts
 - The state of the processor is not known when an interrupt occurs
 - Developing the code is more difficult, as the program must understand and handle any state which can be modified by an interrupt
 - Memory management becomes more complex

When to Use Interrupts

- Not every event is best handled by an interrupt
 - Sometimes it is easier to use polling
 - Sometimes you have sufficient processing resources that the penalty for polling is insignificant
- Interrupts should be used when an event has 3 main characteristics
 - Asynchronous: The event can occur at any time, and there is no way to predict when this will be
 - Urgent: The event needs to be responded to immediately
 - Infrequent: The event is not occurring continuously
- An emergency shutdown event (such as over current) is an example of an event which is a good candidate for using an interrupt
- In contrast, if transmitting a few bytes of data over UART after processing some data, then in this example polling may be used

Interrupts in Your Project



- We need to measure voltage and current at regular time intervals for our calculations
- While taking these measurements we also need to scan and update the display periodically
 - It will be challenging to do both these tasks if we poll the ADC measurements
- We can setup the ADC to auto-trigger on a timer and use ADC conversion complete interrupt
 - While waiting for an ADC measurement we can update the display using a timer interrupt
 - You will need to think about interrupt priorities to meet your timing requirements
- After the calculation are done we can use polling to transmit data over the UART

Interrupt Service Routines

Understanding the Fundamentals

What is an ISR?

- An MCU executes a normal program in an orderly sequence determined by the user
- When an interrupt occurs, the normal program execution is halted, and a special function is executed
 - This function is called the Interrupt Service Routine (ISR)
- An ISR can be thought of as a function with no return value, and no input arguments
 - As always, there are exceptions, but in general this is the case
- The only way for an ISR to communicate with the rest of the program is through variables with shared scope
 - These are global variables with file level or unlimited scope (i.e. static or extern global variables)
- Once the ISR has completed, the processor returns to the code it was executing before the interrupt occurred
- Each interrupt normally has its own ISR
 - These ISRs will be executed in the order of their interrupt priority
- ISRs are not called as part of the normal program execution

How to Use an ISR

```
//An ISR does not take any arguments, and does not return a result
ISR(NAME_vect){
    //Do something
}

void function_1(void){
    //Do something
}

void function_2(void){
    //Do something
}

int main(void){
    while (1){
        //We can call our functions, but NOT an ISR
        function_1();
        function_2();
    }
}
```

ISR Communication

- As the ISR cannot be passed arguments or return a result, it can only communicate with the rest of the program by using global variables with file or unlimited scope
- As the interrupt can occur at any time, any variables written by the ISR can be modified at any point during regular execution
 - Normally the compiler does not expect this, so we must use the *volatile* keyword to alert it
- The *volatile* keyword prevents the compiler from optimizing the code under the assumption the variable will never change
- Failure to use the *volatile* keyword can result in code that crashes or doesn't work

```
volatile uint8_t my_volatile_variable_1;    //Global variable with unlimited scope
extern volatile uint8_t my_volatile_variable_2; //Global variable with unlimited scope
static volatile uint8_t my_volatile_variable_3; //Global variable with file scope

int main(void){
    //Do something
}
```

ISR Communication Example

*//millisecond_counter is not changed by the main function. Therefore, the compiler might infer that the
//millisecond_counter is never updated. In this case, millisecond_counter can never be greater than 1000.
//So the code can be removed (optimized out). The volatile keyword tells the compiler that the variable
//is updated outside the normal program execution, and prevents the code from being removed.*

```
volatile uint16_t millisecond_counter = 0;
```

//This ISR is called by a timer every millisecond

```
ISR(NAME_vect){  
    millisecond_counter++;  
}
```

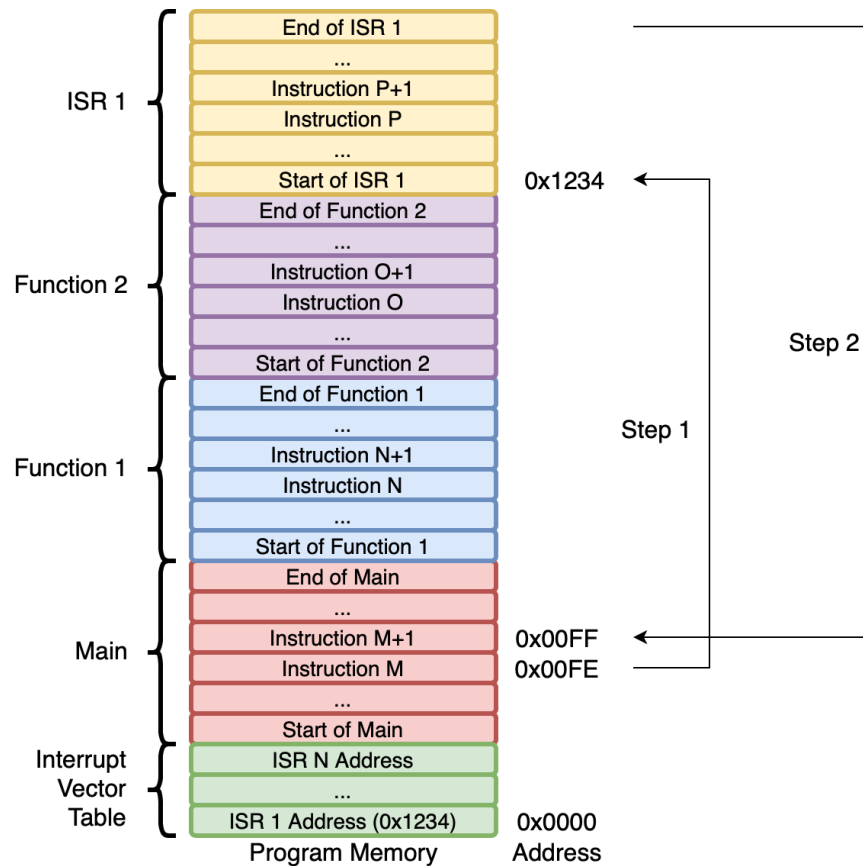
//This program toggles an LED every second

```
int main(void){  
    TIMER_initialise_for_millisecond_interrupt();  
    LED_initialise();  
    while(1){  
        if (millisecond_counter > 1000) {    //Check if 1 second has passed  
            LED_toggle();                  //Toggle the LED  
            millisecond_counter = 0;        //Reset the counter  
        }  
    }  
}
```

ISR Execution (PI)

- The microcontroller has special hardware lines which can be set to signal an interrupt has occurred
- The address of each ISR is stored at a dedicated location in the program memory (usually it is the first thing) in the form of a table
 - This table is called the “Interrupt Vector Table”
 - The position of the interrupt in this table also determines its priority
- When an interrupt occurs, the microprocessor completes the current instruction, and stores the address of the next instruction (i.e. the program counter) on the stack
- Then the address for the ISR of the interrupt that occurred is loaded by the processor to the program counter, and execution of ISR continues from that point
 - In ATmega328P global interrupts are disabled to avoid nested interrupts
- Upon completing the ISR, the address stored on the stack is loaded back into the program counter, and execution resumes from that address
- If other interrupts occurred while executing an ISR, they will be processed in the order of priority after completing the first one

ISR Execution (PII)



Step 1:
Interrupt 1 occurs while instruction M is being executed. Program control is transferred to the address for ISR 1 (0x1234). The next address (0x00FF) is stored in the stack

Step 2:
ISR 1 finishes executing. Program control is returned back to the address which was stored when the ISR was executed (0x00FF)

Interrupts on ATmega328P

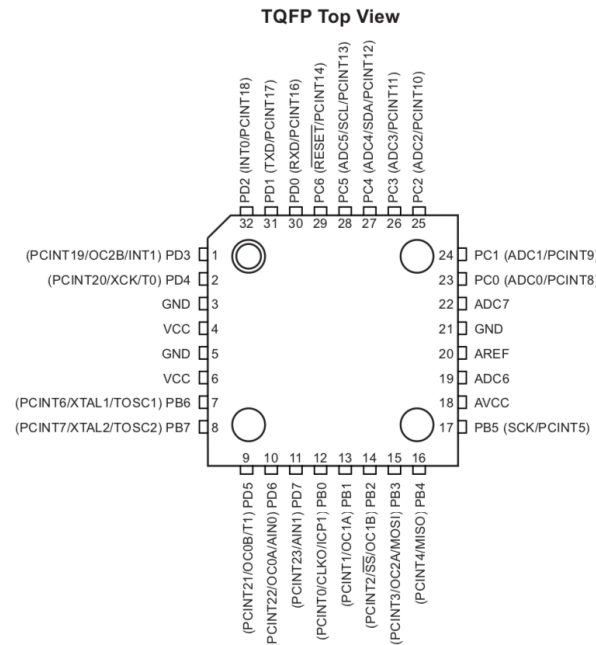
Configuring and Using

ATmega328P Interrupts

- The ATmega328P supports multiple interrupt sources and these include
 - GPIO level changes
 - UART receive complete, transmit buffer empty
 - ADC conversion completion
 - Timer compare match, overflow
- The interrupt priority is fixed where lower addresses in interrupt vector table have a higher priority
- All interrupt sources have their own interrupt enable bit that needs to be set
 - The global interrupt enable bit needs to be also set to enable interrupts
- It takes a minimum of 4 cycles to execute the ISR address in the interrupt vector table
 - The microprocessor has to complete the instruction it was executing and store the contents of general purpose registers, program counter, as well as the status register before servicing an interrupt
- Global interrupts are automatically disabled when an interrupt occurs and enabled when returning from an ISR

ATmega328P Interrupt Pins

- In the ATmega328P the external interrupt pins INT0 and INT1 are PD2 and PD3
- Any of the pin change interrupts (PCINTx) pins can also be used as interrupt sources
 - However they have a shared ISR



ATmega328P Interrupts Registers

- There is one global interrupt enable bit (I-bit) which enables or disables all interrupts

Register	Functionality
SREG	AVR Status Register

- Interrupts associated with most peripherals are configured and controlled using bits in their own registers
- There are usually two bits associated with an interrupt
 - Setting interrupt enable bit causes an interrupt to be triggered by an interrupt event
 - The interrupt flag bit is set whenever an interrupt event occurs regardless of whether the interrupt enable bit is set or cleared
- The interrupt flag bit is cleared automatically when the ISR for the interrupt is executed
 - Alternatively it can be cleared by writing 1 to the flag
- If an interrupt occurs when executing another ISR, the corresponding interrupt flag will be set so that the new interrupt can be serviced after returning from the current ISR

Status Register (SREG)

Bit	7	6	5	4	3	2	1	0
Status	I	T	H	S	V	N	Z	C
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset Value	0	0	0	0	0	0	0	0

- I: *Global Interrupt Enable* must be set for interrupts to be enabled
- The other bits are used by the processor and can be ignored
 - They contain information about the most recently executed instruction by the ALU

The Interrupt Vector Table

Vector No.	Program Address	Source	Interrupt Definition
1	0x0000	RESET	External pin, power-on reset, brown-out reset and watchdog system reset
2	0x0002	INT0	External interrupt request 0
3	0x0004	INT1	External interrupt request 1
4	0x0006	PCINT0	Pin change interrupt request 0
5	0x0008	PCINT1	Pin change interrupt request 1
6	0x000A	PCINT2	Pin change interrupt request 2
7	0x000C	WDT	Watchdog time-out interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 compare match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 compare match B
10	0x0012	TIMER2 OVF	Timer/Counter2 overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 capture event
12	0x0016	TIMER1 COMPA	Timer/Counter1 compare match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 compare match B
14	0x001A	TIMER1 OVF	Timer/Counter1 overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 compare match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 compare match B
17	0x0020	TIMER0 OVF	Timer/Counter0 overflow
18	0x0022	SPI, STC	SPI serial transfer complete
19	0x0024	USART, RX	USART Rx complete
20	0x0026	USART, UDRE	USART, data register empty
21	0x0028	USART, TX	USART, Tx complete
22	0x002A	ADC	ADC conversion complete
23	0x002C	EE READY	EEPROM ready
24	0x002E	ANALOG COMP	Analog comparator
25	0x0030	TWI	2-wire serial interface
26	0x0032	SPM READY	Store program memory ready

Interrupts Associated with Peripherals

- From the interrupt vector table we can see that the peripherals we have discussed so far all have interrupts associated with them
- USART0 has receive complete, transmit data register empty and transmit complete interrupts
 - UCSR0B has the interrupt enable bits and UCSR0A has the interrupt flag bits
- ADC has a conversion complete interrupt
 - ADCSRA has both the interrupt enable and flag bits
- TC0 has overflow, compare match A and compare match B interrupts
 - TIMSK0 has the interrupt enable bits and TIFR0 has the interrupt flag bits
- There are more which you may wish to make use of
 - For example, the zero crossing detector that feeds INT0 can generate an interrupt that could be used to enable taking the set of ADC samples needed for the calculations
- In this lecture, we will investigate the TC0 compare match A interrupt

TIMSK0 Registers

Bit	7	6	5	4	3	2	1	0
TIMSK0	-	-	-	-	-	OCIE0B	OCIE0A	TOIE0
Read/Write	R	R	R	R	R	R/W	R/W	R/W
Reset Value	0	0	0	0	0	0	0	0

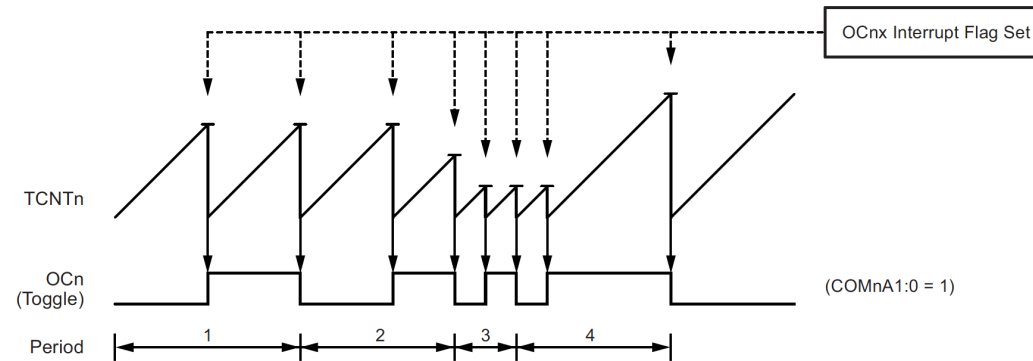
- **OCIE0B:** *Output Compare Match B Interrupt Enable* executes corresponding interrupt when OCR0B match TCNT0
- **OCIE0A:** *Output Compare Match A Interrupt Enable* executes corresponding interrupt when OCR0A match TCNT0
- **TOIE0:** *Overflow Interrupt Enable* executes corresponding interrupt when TCNT0 overflows

TIFR0 Registers

Bit	7	6	5	4	3	2	1	0
TIFR0	-	-	-	-	-	OCFB	OCF0A	TOV0
Read/Write	R	R	R	R	R	R/W	R/W	R/W
Reset Value	0	0	0	0	0	0	0	0

- *OCF0B: Output Compare Match B Flag* is set when OCR0B match TCNT0
- *OCF0A: Output Compare Match A Flag* is set when OCR0A match TCNT0
- *TOV0: Overflow Flag* is set when TCNT0 overflows

Clear Timer on Compare Match



- As we did during the lecture on timers we are going to operate TC0 in CTC mode
- In CTC mode, TCNT0 counts up starting from 0, where TCNT0 is incremented by 1 every timer clock cycle
- TCNT0 value is compared with OCR0A value we will define and when they match, TCNT0 will be set to 0 (i.e. cleared) and OCF0A flag will be set to 1
 - If OCIE0A bit is set a *TIMER0 COMPA* interrupt will be triggered
 - OCF0A flag will be cleared when executing the *TIMER0 COMPA* interrupt

Configuring TCO in CTC Mode

- Lets develop an initialization function to configure TCO to operate in the [CTC mode](#)
- We want TCO to generate an interrupt every 1ms
- Since the ATmega328P uses a 2MHz system clock to achieve this
 - We will use a prescaler of 8 and load 249 to OCR0A
- We also have to enable *TIMER0 COMPA* interrupt by setting the OCIE0A bit
 - We will also start the timer by setting CS0[2..0] to 010

```
//This function configures TCO to generate a TIMER0 COMPA interrupt every 1ms
void tc0_init_1ms_interrupt(void){
    TCCR0A = 0b00000010;    //WGM0[2..0] should be set to 010 for CTC mode
    OCR0A = 249;            //Loading OCR0A with 249 to get a period of 1ms
    TIMSK0 = 0b00000010;    //Enable output compare match A interrupt
    TCCR0B = 0b00000010;    //Initialize CS0[2..0] to 010 so that timer runs with a prescaler of 8
}
```

Writing the ISR

- The C language has no specific rules for interrupts
- Creating the interrupt vector table and placing it in memory is handled by Atmel Studio
 - It also provides macros required for interrupts in `<avr/interrupt.h>`
- An ISR can be declared using the name of the source as listed in the interrupt vector table with `_vect` added at the end
 - In this case we will use `TIMER0_COMPA_vect`
- In the ISR we are going to toggle PB0 while also incrementing `millisecond_counter`, which is a global variable

```
//An ISR is declared ISR(source_name_vect) where source_name is as per interrupt vector table
//TIMER0 COMPA ISR is called when a compare match A occurs
ISR(TIMER0_COMPA_vect){
    PORTB ^= 1 << PINB0;    //Toggle PB0
    millisecond_counter++;    //Increment the global counter variable
}
```

Completing the Code

```
#include <avr/io.h>           //Needed for using the macros for register addresses
#include <avr/interrupt.h>     //Needed for using the macros for interrupts
#include "tc0.h"               //Including our TC0 peripheral library

static volatile uint16_t millisecond_counter = 0; //Declaring a variable that has a file level scope

ISR(TIMERO0_COMPA_vect){      //ISR for TIMERO0 COMPA executes every 1ms
    PORTB ^= 1 << PINB0;     //Toggle PB0
    millisecond_counter++;     //Increment the global counter variable
}

int main(void){
    DDRB |= 1 << PINB5 | 1 << PINB0; //Setting PB0 and PB5 as output pins
    tc0_init_1ms_interrupt();        //Initializing TC0 with a 1ms interrupt
    sei();                           //Enable global interrupts by setting I-bit
    while (1){
        cli();                       //Disable global interrupts to ensure correct operation of if statement
        if(millisecond_counter >= 500){ //Check to see 0.5s has elapsed
            PORTB ^= 1 << PINB5;     //Toggle PB5
            millisecond_counter = 0;  //Reset counter
        }
        sei();                       //Enable global interrupts by setting I-bit
    }
}
```

ISR Rules

- ISRs are usually used to ensure the program responds quickly to certain events
- The processor can only execute one ISR at a time
- Therefore, ensure that your ISR is as short as possible
 - Do not use delays or long loops
 - Do not call expensive functions such as functions used to transmit data over UART
- You should use the interrupt to handle time critical actions, and then set flags to indicate to the main loop that a task needs to be executed
 - For example, use the ISR to read data from the UART, and then set a flag to let the main loop know that there is new data available to process
 - This allows the UART peripheral to receive the next byte

Interrupt Bugs

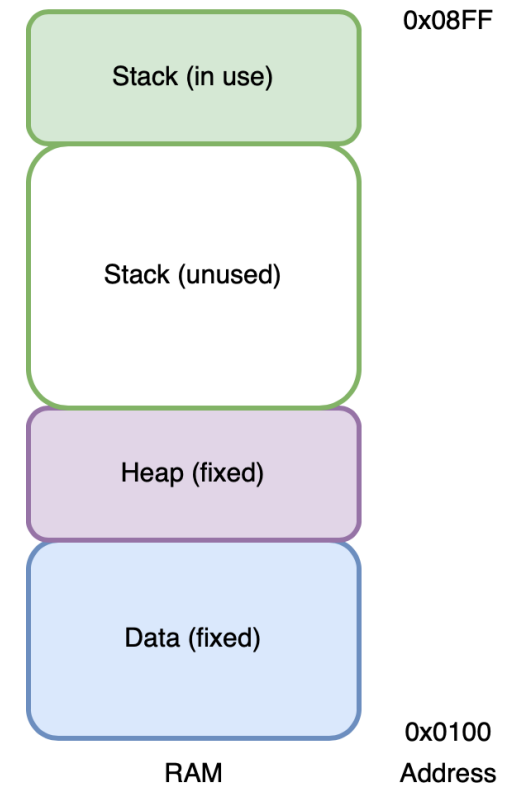
- While interrupts are powerful, interrupting the execution flow of your program can introduce bugs if you are not careful
- We discussed the volatile keyword, which prevents compiler optimizations from introducing bugs
- Suppose your interrupt checks a flag to see if a buffer contains some data, and then transmits that data
 - If the interrupt occurs between the flag getting set and the data being put into the buffer you might transmit data that is not the last thing in the buffer
- Similarly, when a multi-byte variable that is modified by an ISR is also used in a function they need to be atomically read/modified
- Another issue associated with interrupts is stack overflow
 - We will discuss about this briefly in the following section of the lecture
- You will learn about other problems and how to solve them in future courses
- For now, try to keep your interrupt code as simple as possible

RAM Usage

Stack, Heap and Data Segment Spaces

RAM Allocation

- Recall that RAM is one of the three core resources a microcontroller has
- RAM is split into three sections
 - The Stack: Used for temporary storage of variables associated with functions and grows as functions are called, and shrinks as functions return
 - The Heap: Used for storing dynamic variables using malloc and it is set to a fixed size by the programmer
 - The Data Segment: Used for storage of variables with greater than function scope (static or global), which are allocated at compile time and is set to a fixed size depending on how many of these variables need to be stored



RAM Considerations

- The combined size of the stack, heap, and data segment cannot exceed the size of RAM
 - Running out of space is called a “stack collision” or “stack overflow”
- You need to be aware of this in your program
 - If you have unexplained crashes/behavior, you may have a stack collision
- High stack usage can be attributed to
 - Many nested function calls (especially recursion)
 - Allocating large arrays inside functions
- High heap usage can be attributed to
 - Using malloc() to create large dynamic arrays
 - Not releasing allocated memory using free()
- High data usage can be attributed to
 - Allocating large arrays with static or global scope
- Be careful when allocating arrays to store data in your project

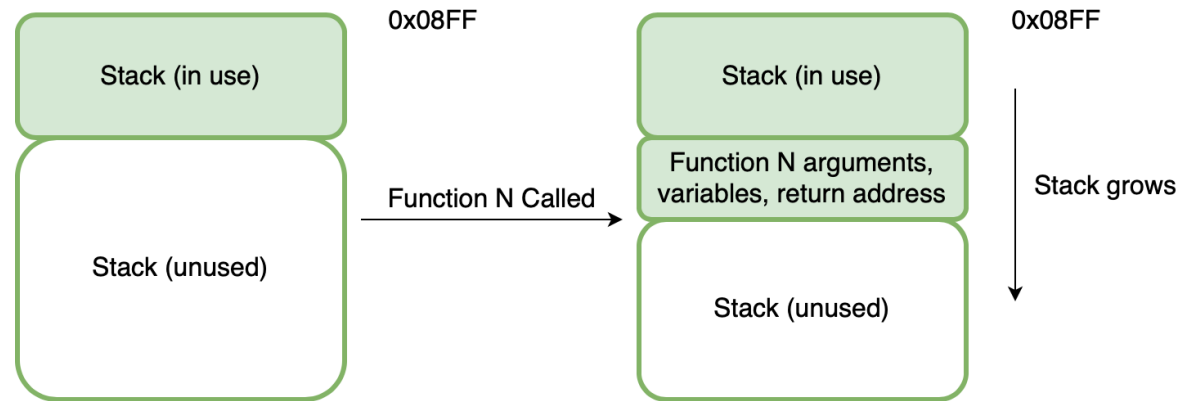
Example: RAM Calculations

A program has two global arrays of 16-bit integers. Each array can store 100 variables. The size of the RAM is 2kB. Assume a 100 byte heap is present.

- What is the total data segment size used by the two arrays?
- How much space is left for the stack?

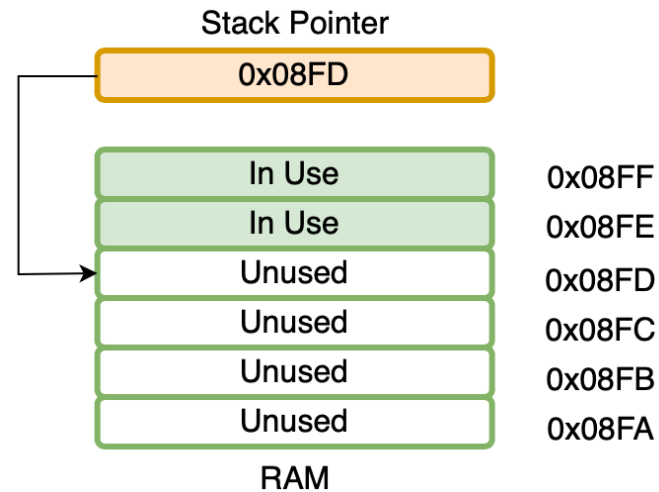
The Stack

- The stack is a consecutive block of data which stores
 - Local variables
 - Function parameters and return values for each function called
 - Return addresses for each function/ISR called
 - Temporary storage of register values
- Usually the stack starts at the last RAM address (highest) and grows down



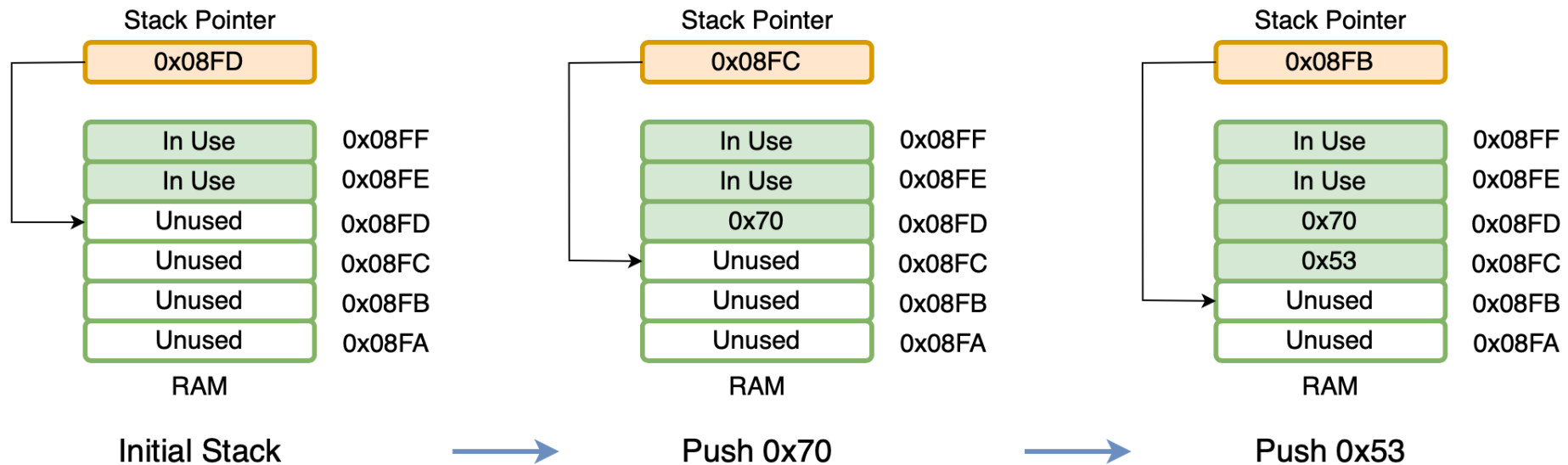
How Does the Stack Operate?

- The stack operates as a last-in-first-out (LIFO) buffer
- The stack pointer stores the address of the next available location on the stack
- The stack pointer is decremented each time data is placed onto the stack (pushed)
- The stack pointer is incremented each time data is removed from the stack (popped)



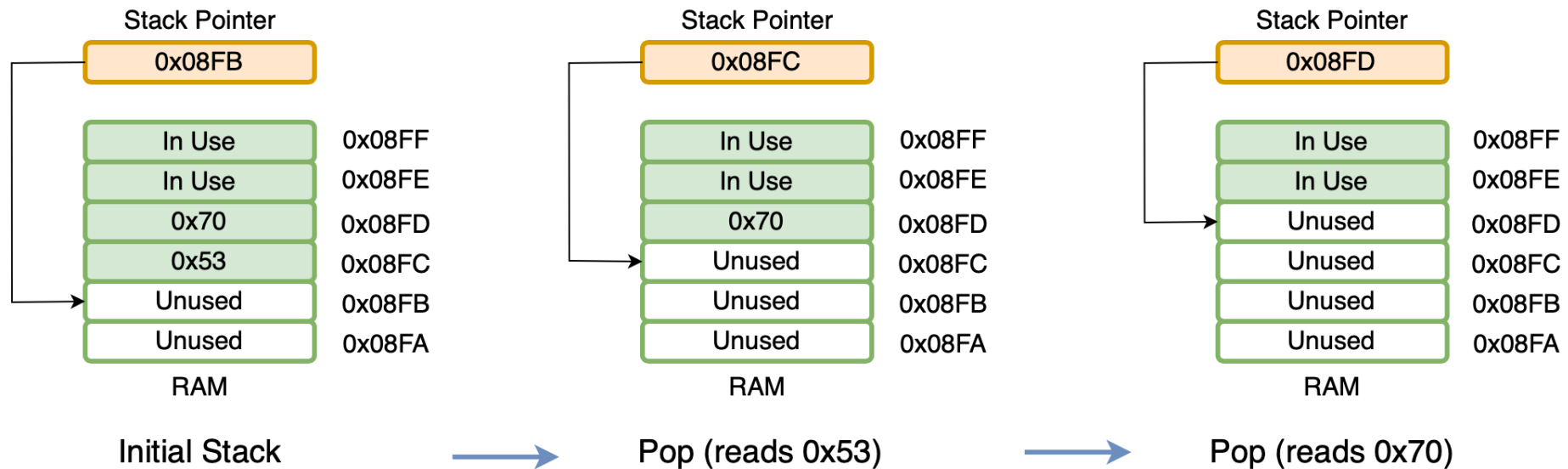
The Push Operation

- To add data to the stack it is “pushed” to the stack pointer address, and the stack pointer is then increased



The Pop Operation

- To remove data from the stack the stack pointer is decreased, and then data is “popped” out from the stack pointer address



Example: Investigating the Use of RAM

Simulate the following code in Atmel Studio and comment on how the RAM is used.

```
#include <avr/io.h>

volatile uint16_t voltages_mv[20];    //Which part of memory is this stored?
volatile uint16_t voltage_rms_mv = 0; //Which part of memory is this stored?

uint16_t my_func(uint16_t data_in){    //How is the function input communicated?
    static uint16_t x;                 //Which part of memory is this stored?
    uint16_t y = 600;
    x = x + data_in + y;
    return x;
}

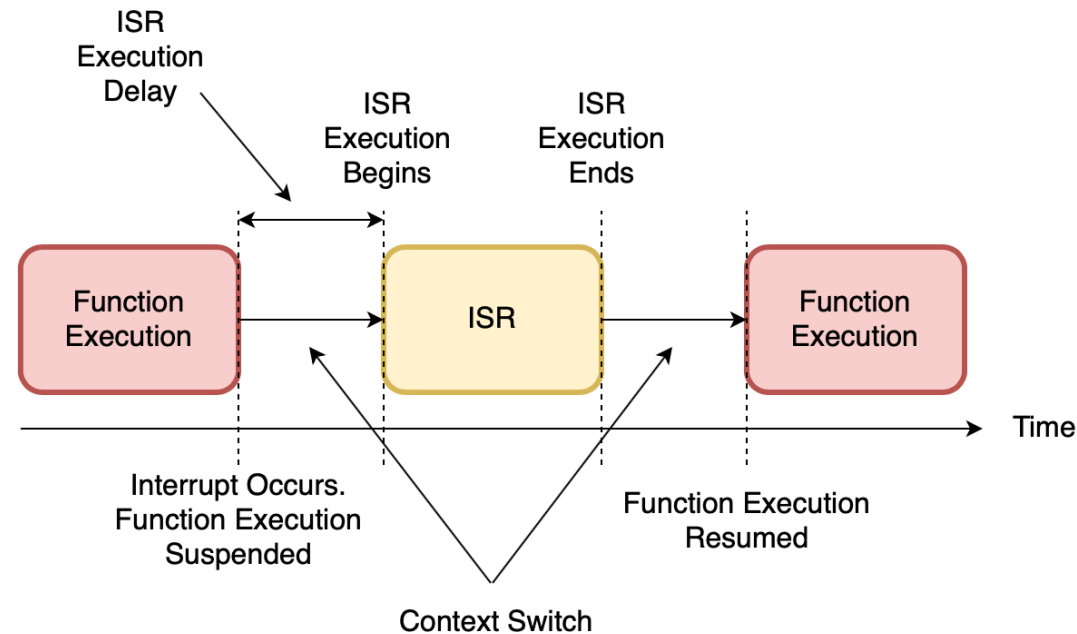
int main(void){
    uint16_t a = 2000;
    uint16_t b = 4000;
    voltage_rms_mv = b/4;              //What happens to the global variable?
    voltages_mv[0] = a+voltage_rms_mv; //What happens to the global variable?
    uint16_t c = my_func(5000);       //What get pushed when jumping and what get popped when returning?
}
```

Context Switching

- While a function is executing (such as during normal program operation) it uses the microprocessor register file
 - The register file is where variables which are being actively used are stored, such as inputs for arithmetic operations, loop counters etc.
 - The ATmega328P has 32 registers
- The ISR will also use the register file, so before executing an ISR the data in the registers that are used must be pushed to the stack
 - You need to make sure that there is sufficient space in the stack
- In addition, the program counter and the status register also needs to be pushed to the stack
- After the ISR completes, these values are loaded back from the stack
- This process is called a “context switch”
 - The compiler automatically adds these instructions to the ISR

ISR Execution Delay

- The time taken to start executing an ISR is limited by how long the context switch takes
 - This delay time is an important design parameter



Acknowledgments

These slides are adapted from material prepared by Travis Scott & Muhammad Nadeem